

Linear Regression From Scratch: Derivation to Python Implementation

1. What is Linear Regression?

Linear regression is a supervised learning algorithm that learns a relationship between:

- Input variables/features (x)
- Output/target variable (y)

The goal is to find a function that predicts $\hat{y}$ as accurately as possible.

For **simple linear regression**:

$$\hat{y} = mx + b$$

where:

- (m) = slope (weight)
- (b) = bias/intercept
- ($\hat{y}$) = predicted output

Example:

Hours studied	Marks
1	50
2	55
3	65
4	70

The model tries to learn:

$$marks = m(hours) + b$$

2. Prediction Error

For every training example:

$$error = y - \hat{y}$$

Example:

Actual:

$$y = 80$$

Prediction:

$$\hat{y} = 75$$

Error:

$$80 - 75 = 5$$

The model wants these errors to become as small as possible.

3. Mean Squared Error (MSE)

A single error is not enough. We calculate the average squared error.

$$J(m, b) = \frac{1}{n} \sum_{i=1}^n (y_i - (mx_i + b))^2$$

Where:

- J = cost function
- n = number of training examples
- y_i = actual value
- $(mx_i + b)$ = predicted value

The objective:

$$\text{Minimize } J(m, b)$$

4. Gradient Descent

Gradient descent is an optimization algorithm used to minimize the cost function.

The update rule is:

$$parameter = parameter - \alpha \times gradient$$

where:

- α = learning rate
- gradient = direction and amount of change

For our model:

$$m = m - \alpha \frac{\partial J}{\partial m}$$

$$b = b - \alpha \frac{\partial J}{\partial b}$$

5. Deriving the Gradient for Slope (m)

Starting with:

$$J(m, b) = \frac{1}{n} \sum_{i=1}^n (y_i - (mx_i + b))^2$$

We want:

$$\frac{\partial J}{\partial m}$$

Step 1: Move constants outside

Since:

$$\frac{1}{n}$$

is a constant:

$$\frac{\partial J}{\partial m} = \frac{1}{n} \sum_{i=1}^n \frac{\partial}{\partial m} (y_i - (mx_i + b))^2$$

Step 2: Apply chain rule

Let:

$$u = y_i - (mx_i + b)$$

Then:

$$u^2$$

Using chain rule:

$$\frac{d}{dm}(u^2) = 2u \frac{du}{dm}$$

Step 3: Differentiate the inside

$$u = y_i - mx_i - b$$

Taking derivative with respect to (m):

$$\frac{du}{dm} = -x_i$$

because:

- (y_i) is constant $\rightarrow$ derivative is 0
 - (b) is constant $\rightarrow$ derivative is 0
 - (mx_i) becomes (x_i)
-

Step 4: Substitute

$$2(y_i - (mx_i + b))(-x_i)$$

Therefore:

$$\boxed{\frac{\partial J}{\partial m} = -\frac{2}{n} \sum_{i=1}^n x_i (y_i - (mx_i + b))}$$

6. Deriving the Gradient for Bias (b)

Again:

$$J(m, b) = \frac{1}{n} \sum_{i=1}^n (y_i - (mx_i + b))^2$$

Differentiate with respect to (b):

$$\frac{\partial J}{\partial b}$$

Inside:

$$y_i - mx_i - b$$

Derivative:

$$\frac{d}{db}(y_i - mx_i - b) = -1$$

Therefore:

$$\frac{\partial J}{\partial b} = -\frac{2}{n} \sum_{i=1}^n (y_i - (mx_i + b))$$

7. Gradient Descent Algorithm

The complete process:

```
Initialize m and b Repeat: 1. Predict values y_hat = mx + b 2. Calculate error 3.
Calculate gradients 4. Update m and b Until convergence
```

8. Simple Linear Regression

Implementation

```
import numpy as np
import matplotlib.pyplot as plt

# Training data

x = np.array([1, 2, 3, 4, 5])
y = np.array([3, 5, 7, 9, 11])

# Initialize parameters

m = 0
b = 0

learning_rate = 0.01
epochs = 1000

n = len(x)

for epoch in range(epochs):

    # Prediction

    y_pred = m*x + b

    # Error

    error = y - y_pred

    # Gradients

    dm = -(2/n) * np.sum(x * error)

    db = -(2/n) * np.sum(error)

    # Update parameters

    m = m - learning_rate * dm
```

```
b = b - learning_rate * db

print("Slope:", m)
print("Bias:", b)

# Plot

plt.scatter(x, y)

plt.plot(
    x,
    m*x+b,
    color="red"
)

plt.show()
```

9. Multiple Linear Regression

Simple regression has one feature:

$$\hat{y} = mx + b$$

Multiple regression has multiple features:

$$\hat{y} = w_1x_1 + w_2x_2 + w_3x_3 + b$$

Example:

House price prediction:

$$price = w_1(size) + w_2(bedrooms) + w_3(age) + b$$

10. Gradient Derivation for Multiple Regression

For any weight:

$$w_j$$

the derivative is:

$$\frac{\partial J}{\partial w_j} = -\frac{2}{n} \sum_{i=1}^n x_{ji}(y_i - \hat{y}_i)$$

Bias:

$$\frac{\partial J}{\partial b} = -\frac{2}{n} \sum_{i=1}^n (y_i - \hat{y}_i)$$

Updates:

$$w_j = w_j - \alpha \frac{\partial J}{\partial w_j}$$

$$b = b - \alpha \frac{\partial J}{\partial b}$$

11. Multiple Linear Regression Without Matrices

We update each weight using a loop.

```
import numpy as np

# Features

X = np.array([
    [1500,3,10],
    [2000,4,5],
    [1200,2,20],
    [1800,3,8]
])

# Target

y = np.array([
    500000,
    700000,
    400000,
    600000
])

number_of_features = X.shape[1]

weights = np.zeros(number_of_features)

b = 0

learning_rate = 0.0000001

epochs = 1000

n = len(y)

for epoch in range(epochs):

    # Predictions

    predictions = []

    for row in X:
```

```
prediction = b

for j in range(number_of_features):

    prediction += weights[j] * row[j]

predictions.append(prediction)

predictions = np.array(predictions)

# Error

error = y - predictions

# Update weights

for j in range(number_of_features):

    gradient = -(2/n) * np.sum(
        X[:,j] * error
    )

    weights[j] -= learning_rate * gradient

# Update bias

b_gradient = -(2/n) * np.sum(error)

b -= learning_rate * b_gradient

print("Weights:", weights)

print("Bias:", b)
```

12. What Happens During Training?

Every epoch:

Step 1: Prediction

$$\hat{y} = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

Step 2: Error

$$error = y - \hat{y}$$

Step 3: Calculate gradient

For every feature:

$$gradient = -\frac{2}{n} \sum x_j(y - \hat{y})$$

Step 4: Update weight

$$w_j = w_j - \alpha(gradient)$$

Step 5:

Repeat.

13. Simple vs Multiple Linear Regression

	Simple Linear Regression	Multiple Linear Regression
Features	1	Many
Equation	$\hat{y} = mx + b$	$\hat{y} = w_1x_1 + w_2x_2 + \dots + b$
Weights	1	Many
Gradient calculations	1	One per feature
Implementation	Direct	Loop over features

14. Key Takeaways

The cost function

$$J = \frac{1}{n} \sum (y - \hat{y})^2$$

Gradient descent rule

$$parameter = parameter - \alpha gradient$$

Simple regression gradients

$$\frac{\partial J}{\partial m} = -\frac{2}{n} \sum x(y - \hat{y})$$

$$\frac{\partial J}{\partial b} = -\frac{2}{n} \sum (y - \hat{y})$$

Multiple regression gradient

$$\frac{\partial J}{\partial w_j} = -\frac{2}{n} \sum x_j(y - \hat{y})$$

The important idea:

Linear regression with gradient descent is simply repeatedly adjusting parameters in the direction that reduces the error.

The same principle extends to much larger machine learning models. The difference is only the number of parameters being updated.